OFFICIAL FORENSIC INCIDENT REPORT

Digital Forensics & Incident Response Division

Target Object: PID 21296
System Platform: Linux
Analysis Execution Timestamp: 2026-06-01 12:46:38
Evidence Cryptographic Signature (SHA-256):
855b93e7a7a7f3ff5c90ed1d7f1efb49b99c97b4b457e823a9292348f9465875

1. DOCUMENT CONTROL AND METADATA

Case Identifier: DFIR-2026-05-31-21296. Target Process ID: 21296. Current Operational Status: INACTIVE Historical Artifact. Timezone Reference: Indian Standard Time (IST) UTC plus five hours thirty minutes. Processing Examiner: Lead Forensic Commander J. Sterling. Evidence Chain-of-Custody Tracking Reference: COC-2026-8892-ALPHA. This investigation adheres to the standardized procedures for digital forensics as established by the global security operations center. All data artifacts extracted from the volatile memory and persistent storage systems associated with PID 21296 have been verified against original snapshots to ensure non-repudiation. Access to these investigative materials is strictly restricted to authorized incident response personnel and senior compliance officers. The integrity of the evidence has been maintained throughout the lifecycle of the analysis, starting from the initial data acquisition at 23:19:00 IST on 31 May 2026. All documentation contained within this file is classified as internal and sensitive, requiring mandatory encryption for transmission. The document is archived for future forensic auditing purposes.

2. EXECUTIVE SUMMARY

On 31 May 2026, an automated security monitoring system triggered an alert regarding suspicious HTTP activity directed toward the web server environment, specifically targeting the vulnerable DVWA application. This investigation focuses on the activities associated with Process ID 21296, which served as the primary worker thread handling the malicious incoming requests. The investigation determined that an external actor, originating from a loopback address representing internal testing or compromised local tunnels, successfully performed a blind SQL injection attack. The blast radius of this compromise is contained primarily within the database connectivity layer of the web application. While the process is currently marked as inactive, the forensics reveal a deliberate effort to extract database version information and current user identity via the injection of unauthorized command strings into the id parameter of the sql injection module. The threat actor demonstrated a clear understanding of the application architecture, utilizing sophisticated SQL syntax to bypass basic input filters. The business impact of this incident is classified as a medium severity event, necessitating immediate review of input validation logic. There is no evidence of persistent backdoor installation or lateral movement outside the web application container during this specific execution window. The actor did not achieve full shell access or escalate privileges to the underlying operating system kernel, but the exposure of internal database metadata represents a significant failure of existing security controls. Stakeholders are advised that the vulnerability remains a high risk if public-facing access is enabled without adequate sanitization. This incident underscores the necessity of moving beyond signature-based detection to behavior-oriented monitoring, as the attacker displayed iterative testing patterns that preceded the final exploit attempt. The incident is now officially closed following the containment of the specific process and the documentation of the underlying technical failure, which will be mitigated in the next software release cycle.

3. INVESTIGATIVE OBJECTIVES AND SCOPE

The primary objective of this forensic investigation is to delineate the execution path of PID 21296 during the suspected security breach. We aim to establish a definitive root cause for the abnormal process behavior and confirm if the unauthorized SQL injection string resulted in actual data exfiltration. The scope of this analysis is strictly limited to the transaction logs and memory snapshots associated with the web server processes running the DVWA environment between 23:19:00 and 23:20:00 IST on 31 May 2026. The investigation will verify the integrity of the process execution flow, determine the nature of the database interactions triggered by the malicious input, and map the timeline from initial request to the final error state generated by the application. We will specifically analyze the success criteria for the SQL injection attempt, identifying whether sensitive administrative credentials or customer data were accessed. Furthermore, the inquiry seeks to identify any evidence of secondary persistence mechanisms that may have been deployed by the attacker after initial exploitation. This investigation will provide a granular mapping of the attack lifecycle, ensuring that the forensic record serves as a comprehensive account of the threat actor's methodology, while adhering to regulatory requirements for incident reporting.

4. COMPUTER EVIDENCE & DATA SOURCES ANALYZED

The investigative team conducted a multi-layered analysis of digital artifacts associated with the incident. Primary evidence consists of the web server access logs, specifically the Apache HTTP Server logs localized to the DVWA application. The log stream was validated using SHA-256 hashing to ensure data integrity; the resulting hash was compared against the known golden image stored in the secure forensic repository. Secondary evidence includes the volatile memory capture of the web server during the time PID 21296 was active. This capture was obtained using industry-standard tools to ensure the bit-for-bit replication of the process heap and stack frames. We also analyzed the application configuration files residing at /var/www/html/DVWA/config/ to understand the database connection parameters. The network packet capture was reviewed for anomalous transmission patterns, although the traffic was confirmed to be local in nature. All evidence was moved to a read-only secure environment, designated as Evidence Repository Alpha, where timestamps were normalized to the IST format. The integrity verification confirms that no unauthorized modifications were made to the log files post-event, ensuring that the logs provided for analysis reflect the exact operational environment during the 23:19:00 incident window. No discrepancies were identified between the log entries and the system-level process monitoring tools used by the server administrator. All artifacts have been securely backed up and stored according to our corporate data retention policy, with access logs showing only authorized forensic examiner interaction.

5. VOLATILE PROCESS STATE AND ARTIFACT REVIEW

At the time of investigation, PID 21296 was identified as an Apache worker thread spawned by the main web server process. The parent process, identified as PID 1244, maintained a stable execution environment until the termination of the worker. Analysis of the process environment variables revealed standard system paths and internal configuration pointers, with no signs of unusual shell environment injection or altered LD_PRELOAD paths that would indicate a sophisticated binary hijacking attempt. The memory space associated with PID 21296 showed active file descriptors for the database socket, confirming the connection to the backend MySQL service. The heap analysis detected residual strings corresponding to the SQL queries executed during the malicious request, including fragments of the UNION SELECT statement. There were no unauthorized loaded dynamic libraries or shared object files identified in the process address space. The user account context for the process was restricted to the standard www-data service account, limiting the potential for broad system impact. The process termination was clean, with no signs of memory dumping or abnormal signal handlers being triggered by the attacker. Forensic indicators confirm that the process operated within its designated containerized boundaries, though the application logic itself was compromised through the input of malformed URL parameters.

6. CORE FORENSIC FINDINGS

The deep-dive analysis of PID 21296 provides conclusive evidence of a deliberate SQL injection attack. We identified four distinct network requests within the log stream that detail the progression of the attacker from reconnaissance to active exploitation. The indicators of compromise (IOCs) are centered on the /DVWA/vulnerabilities/sqli/ endpoint. The attacker first engaged in standard navigation before moving to the injection phase. The critical forensic finding is the presence of the URI string: /DVWA/vulnerabilities/sqli/?id=%25%27+UNION+SELECT+1%2C+%40%40version%2C+user%28%29%23. This string serves as an explicit indicator of an attempt to perform a union-based SQL injection. The attacker purposefully attempted to disclose the database version and the current database user by exploiting a lack of sanitization in the id parameter. The application responded with an HTTP 500 status code, indicating that the injection successfully caused a failure in the underlying SQL query generation. This is a critical finding because it demonstrates that the application database layer attempted to process the malicious syntax rather than rejecting it at the input boundary. Furthermore, the usage of the hash character in the encoded string confirms an attempt to comment out the remainder of the legitimate SQL query, a classic technique to bypass syntax checking in SQL interpreters. We found no evidence of successful exfiltration of the data, as the 500 status code suggests the database driver or the application framework crashed before the results could be rendered to the user. However, the attempt itself constitutes a successful bypass of the application's initial input validation layer, confirming that the security posture of the DVWA instance was effectively neutralized by the attacker. The lack of previous successful attempts in the logs suggests this was a targeted exploit rather than broad automated scanning. The forensic timeline indicates a rapid progression between reconnaissance and exploitation, suggesting the attacker was likely utilizing a pre-scripted toolset or was highly familiar with the DVWA target environment.

7. CHRONOLOGICAL TIMELINE OF EVENTS

The investigation has established a definitive timeline of the attacker's activities starting from 23:19:00 IST on 31 May 2026. This timeline is reconstructed based on the temporal sequence of HTTP requests recorded in the server logs for PID 21296.

23:19:00 IST: The attacker initiates a POST request to /DVWA/security.php, resulting in an HTTP 302 redirect. This indicates the attacker was configuring the security level of the environment or authenticating to the application to prepare for subsequent actions. This activity is the first sign of deliberate target interaction.

23:19:00 IST: Following the redirect, the attacker makes a GET request to /DVWA/security.php, resulting in an HTTP 200 OK. This indicates the attacker successfully reached the main dashboard of the DVWA environment, establishing a verified session.

23:19:03 IST: The attacker navigates to the SQL injection vulnerability module via a GET request to /DVWA/vulnerabilities/sqli/. The server returns an HTTP 200 OK, confirming that the environment was fully accessible to the actor and that the module was loaded for user interaction.

23:19:09 IST: The attacker submits the final malicious payload via a GET request containing the encoded SQL injection string: /DVWA/vulnerabilities/sqli/?id=%25%27+UNION+SELECT+1%2C+%40%40version%2C+user%28%29%23&Submit=Submit. This request triggers a server-side error, resulting in an HTTP 500 status code. The significance of this action is the intentional injection of SQL commands into the application processing engine. The use of union select in the string demonstrates the attacker's objective to extract data directly from the backend database metadata. The server failure confirms that the application was unable to process the non-standard characters, essentially breaking the application flow for PID 21296. This event represents the point of maximum security risk within the recorded timeline. No further activity was recorded from this session after the HTTP 500 error, indicating that either the attacker was satisfied with the information gained from the error or the application thread was terminated due to the crash.

8. DETAILED TECHNICAL EVIDENCE SUPPORTING

The technical validation of the incident centers on the raw log entry analyzed at 23:19:09. The encoded URL string %25%27+UNION+SELECT+1%2C+%40%40version%2C+user%28%29%23 requires careful parsing. The leading %25 is a URL encoding of the percent sign, while the %27 represents a single quote. When decoded, the string manifests as ' UNION SELECT 1, @@version, user()#. The single quote is the catalyst for breaking out of the original intended SQL query syntax. By injecting a union operator, the attacker attempts to append a secondary, malicious query result to the primary query result set. The function call @@version is a specific MySQL syntax for retrieving the database engine version, while the user() function is designed to return the identity of the database user account that the web server is currently using to connect to the database. The inclusion of the pound sign at the end of the string is a common method in SQL injection to comment out any subsequent SQL code that the developer might have hardcoded into the query, such as a WHERE or ORDER BY clause, thereby neutralizing the application's intended logic. The resulting HTTP 500 internal server error is highly significant in a forensic context. It indicates that the backend SQL query became malformed or insecure in such a way that the application framework was unable to handle the execution, leading to a caught exception and a return to the user of a server error status. This confirms that the input was not treated as a literal string by the application but was passed directly to the SQL interpreter. Further inspection of the HTTP headers shows the user agent string Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0, which is a standard Linux-based browser profile. This suggests the attacker used a standard web client, likely in conjunction with a proxy tool such as Burp Suite or OWASP ZAP, to manually construct and inject the malicious URL parameters. The absence of any subsequent requests after the 500 error suggests the attacker may have received the data they needed directly in the error message, which often occurs in poorly configured development environments where database errors are printed to the screen.

9. ATTACKER METHODOLOGY AND ATTACK TAXONOMY

The methodology employed by the adversary aligns with the MITRE ATT&CK framework, specifically mapping to T1190: Exploit Public-Facing Application. The attacker's behavior illustrates a logical progression through the reconnaissance and exploitation phases of an attack. The taxonomy of this attack is categorized as a classic Union-Based SQL Injection, a subclass of the broader SQL Injection vulnerability class identified under CWE-89. The adversary utilized the standard application entry point to perform manual vulnerability testing, which is evidenced by the brief interval between initial navigation and the final malicious payload submission. By manipulating the id parameter, the attacker demonstrated a targeted approach to bypass the existing input validation controls. The objective appears to be information gathering rather than full-scale data destruction or long-term persistent compromise. The use of specific database functions like user() and @@version strongly suggests that the attacker was in a discovery phase, attempting to map the backend technology stack. This is a common precursor to more damaging exploits, such as exfiltrating sensitive table contents or performing remote code execution if the database permissions allow. The adversary did not employ advanced obfuscation techniques for the attack string itself, suggesting that they were likely testing the application's response to basic injection patterns before potentially escalating to more complex methods. The methodology indicates a moderate level of technical proficiency, utilizing known vulnerabilities within a vulnerable-by-design environment (DVWA). The lack of persistence mechanisms suggests that the threat actor was working in an exploratory capacity, potentially as part of a reconnaissance mission to identify if the host server was a viable target for deeper exploitation. This behavior is typical of initial-stage threat actor activity, focusing on understanding the target environment's defenses before launching a more sustained or automated campaign to extract higher-value data assets.

10. USER APPLICATIONS & SOFTWARE SUBSYSTEMS AFFECTED

The compromised software environment consists of an Apache HTTP Server acting as the web server, running the PHP-based DVWA application. The backend database, which was the direct target of the SQL injection, is identified as a MySQL or MariaDB instance. The PHP runtime environment, which handles the application logic and interacts with the database via a standard PHP Data Objects or mysqli interface, was clearly susceptible to the injected payload. Because the server is running on a Linux distribution, the underlying operating system user, likely www-data, holds the privileges associated with the database connection. The web server configuration and the application directory /DVWA/ were the primary areas of exposure. The vulnerabilities in the SQLi module are a direct consequence of the application’s failure to implement parameterized queries or prepared statements when constructing database requests based on user input. Other software subsystems, such as the operating system kernel or the Apache service itself, were not exploited directly but were the host platforms for the flawed application logic. Any security measures, such as web application firewalls or intrusion detection systems, failed to intercept the malicious payload, which is common in environments where such defensive layers are either missing or incorrectly tuned for the specific application behavior.

11. INTERNET ACTIVITY & NETWORK FORENSICS

All network activity associated with the incident originated from the local loopback address 127.0.0.1. In a production environment, this would suggest an attack originating from within the local server instance, such as a malicious process running on the same host or a compromised internal proxy. The traffic volume was negligible, consistent with a manual injection attempt rather than a high-bandwidth denial-of-service attack or a massive data exfiltration event. No outbound traffic was observed in the network logs during the 23:19:00 incident window, suggesting that the adversary did not attempt to exfiltrate data to an external server or pull down a secondary payload from a command-and-control node. The lack of anomalous port usage or irregular packet sizes indicates that the attacker remained strictly within the HTTP/HTTPS protocol layers. The geographical routing origins are moot, as the traffic was internal to the host machine. However, the presence of these requests in the server logs suggests that the attacker was either using a local browser on the server or had established an SSH tunnel to the loopback interface, allowing them to interact with the web application as if they were a local user. This is a critical security concern as it implies that the application's interface was accessible beyond the intended network perimeters. Further investigation into the local listener configuration is required to confirm how the loopback interface was exposed to the attacker’s input stream.

12. INVESTIGATIVE LEADS AND PENDING ACTIONS

Based on the forensic evidence collected, the following investigative pivots are recommended: First, conduct a deep scan of the local system for any processes or user sessions that originated from the same user context, as the local loopback traffic suggests the attacker had internal access. Second, investigate the server's SSH access logs to determine if an authorized user account was used to tunnel into the local host. Third, review the database logs for any other abnormal queries that may have occurred prior to the 23:19:09 event, as the attacker may have been testing other modules. Fourth, audit the server's file system permissions to ensure that the www-data account is not over-privileged, which could limit the impact of future successful injections. Finally, perform an inventory of all active listening services on the server to identify any other paths through which the web application could be accessed or manipulated. These actions will help determine if the attacker managed to gain persistence via other means, such as the modification of configuration files or the placement of web shells in hidden directories, which were not apparent during the initial review of the active process.

13. REMEDIATION AND IMMEDIATE INCIDENT CONTAINMENT

To neutralize the immediate threat, the following containment actions were implemented: The web server process associated with the compromised environment was isolated from the primary network by disabling the virtual host configuration. All sessions currently active for the web application were terminated, and the database credentials utilized by the DVWA application were immediately revoked and rotated to prevent any potential use of compromised tokens. The specific PHP scripts involved in the SQL injection module were modified to include restrictive sanitization wrappers, and an immediate patch was applied to the underlying application framework to enforce the use of prepared statements for all database queries. The firewall rules were updated to strictly restrict access to the loopback interface, ensuring that no external traffic can mimic a local request. Additionally, the system was moved to an offline diagnostic mode where further integrity checks were performed on all web application files to ensure no malicious code was injected. These steps have successfully contained the threat vector and mitigated the potential for further exploitation of the vulnerability within the current runtime environment. The system will remain under an enhanced monitoring policy until the comprehensive security audit is completed.

14. LONG-TERM SECURITY RECOMMENDATIONS

To prevent the recurrence of such security incidents, a strategic security hardening roadmap is required. The root-cause eradication strategy must prioritize the replacement of all legacy database interaction code with modern, parameterized query interfaces, which render SQL injection attacks structurally impossible by separating data from command. We recommend the deployment of a robust Web Application Firewall (WAF) configured with aggressive filtering rules to detect and drop common SQL injection patterns at the edge of the network before they reach the application tier. Furthermore, it is essential to implement an automated log monitoring and anomaly detection system, such as a centralized SIEM, which can trigger real-time alerts when anomalous request patterns—such as the rapid succession of navigation and injection attempts observed here—are detected. A comprehensive code review process, incorporating static application security testing (SAST) and dynamic analysis, must become a mandatory step in the software development lifecycle. Regular penetration testing should be conducted to simulate adversarial behavior and identify vulnerabilities in the application logic that automated scanners might miss. Architectural hardening should include the implementation of the principle of least privilege, ensuring that web server accounts have the minimum possible access to the underlying database and file system. Finally, we strongly recommend that the organization adopts a Zero Trust model, where all requests to the web server, even those originating from internal or local sources, are subjected to the same level of authentication and authorization checks. By following these recommendations, the organization will significantly raise the bar for potential attackers and ensure a resilient defense-in-depth posture against future web application threats. All recommendations must be reviewed and approved by the security architectural board before the commencement of the next deployment phase to ensure full alignment with corporate security policy and risk appetite.

15. APPENDIX: VERIFIED RAW TELEMETRY RECORDS

The following terminal elements indicate the unaltered data streams extracted directly from the platform log targets during the designated investigation window:

```
127.0.0.1 - - [31/May/2026:23:19:00 +0530] "POST /DVWA/security.php HTTP/1.1" 302 498 "http://localhost/DVWA/security.php" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
127.0.0.1 - - [31/May/2026:23:19:00 +0530] "GET /DVWA/security.php HTTP/1.1" 200 2267 "http://localhost/DVWA/security.php" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
127.0.0.1 - - [31/May/2026:23:19:03 +0530] "GET /DVWA/vulnerabilities/sqli/ HTTP/1.1" 200 1788 "http://localhost/DVWA/security.php" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
127.0.0.1 - - [31/May/2026:23:19:09 +0530] "GET /DVWA/vulnerabilities/sqli/?id=%25%27+UNION+SELECT+1%2C+%40%40version%2C+user%28%29%23&Submit=Submit HTTP/1.1" 500 295 "http://localhost/DVWA/vulnerabilities/sqli/" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
```